

voice-postgres

Harbor & Bean

Cafe operations database — reference guide

[Schema](#) · [seed catalog](#) · [voice tools](#) · [example SQL](#)

1. Overview

Harbor & Bean is a neighborhood cafe. This database is created by the voice-postgres project (`sql/01_schema.sql` and `sql/02_seed.sql`) and is what the xAI Voice Agent queries through custom function tools.

Connect

```
postgresql://voice:voice@127.0.0.1:55432/voice_postgres
```

Docker Compose publishes that port on loopback only. The companion console is `python scripts/db_console.py` after `source ./env.sh`.

What lives here

- 7 tables: customers, employees, products, orders, order_items, shifts, inventory_adjustments
- 3 views: v_order_totals, v_low_stock, v_daily_sales
- Seed: 12 guests, 6 staff, 15 SKUs, ~23 tickets, shifts for yesterday / today / tomorrow

2. Relationships

Orders belong to a customer and optionally an employee. Line items snapshot `unit_price` at purchase time. Inventory adjustments are an audit log; `products.stock_qty` is the live on-hand count.

From	Column	To	On delete
orders	customer_id	customers.id	restrict
orders	employee_id	employees.id	nullable / restrict
order_items	order_id	orders.id	CASCADE
order_items	product_id	products.id	restrict
shifts	employee_id	employees.id	restrict
inventory_adjustments	product_id	products.id	restrict

*Ticket flow: **pending** → **preparing** → **ready** → **completed** (or **cancelled** from any open state).*

3. Tables

customers

Cafe guests with loyalty accounts. Email is unique.

Column	Type	Notes
id	serial PK	
name	text not null	
email	text unique not null	Lookup key for create_order
phone	text	nullable
loyalty_points	int not null default 0	
created_at	timestampz default now()	

employees

Staff. *role* ∈ *barista* | *baker* | *shift_lead* | *manager*.

Column	Type	Notes
id	serial PK	
name	text not null	
role	text not null	checked enum
hourly_rate	numeric(6,2) not null	
hired_on	date not null	
active	boolean default true	

products

Menu and merch. *category* ∈ *coffee* | *tea* | *pastry* | *food* | *merchandise*. *stock_qty* is on-hand.

Column	Type	Notes
id	serial PK	
sku	text unique not null	e.g. LAT-OAT
name	text not null	
category	text not null	checked enum
unit_price	numeric(8,2) not null	≥ 0
stock_qty	int not null default 0	≥ 0
reorder_at	int not null default 5	low-stock threshold
active	boolean default true	

orders

Customer tickets. *status* ∈ *pending* | *preparing* | *ready* | *completed* | *cancelled*.

Column	Type	Notes
id	serial PK	
customer_id	int not null FK	→ customers
employee_id	int FK	→ employees, nullable
status	text not null default pending	checked enum
notes	text	nullable
placed_at	timestampz default now()	indexed
completed_at	timestampz	set on completed/cancelled

order_items

Line items. `unit_price` is copied from the product at order time.

Column	Type	Notes
id	serial PK	
order_id	int not null FK	→ orders CASCADE
product_id	int not null FK	→ products
quantity	int not null	> 0
unit_price	numeric(8,2) not null	snapshot

shifts

Who is scheduled, on which day and station (bar, register, kitchen, floor).

Column	Type	Notes
id	serial PK	
employee_id	int not null FK	→ employees
shift_date	date not null	indexed
start_time	time not null	
end_time	time not null	
station	text not null default bar	

inventory_adjustments

Audit log of stock changes (receiving, waste, voice-agent edits and sales).

Column	Type	Notes
id	serial PK	
product_id	int not null FK	→ products
delta	int not null	signed quantity
reason	text not null	
created_at	timestampz default now()	

4. Views

v_order_totals

One row per order: `order_id`, `customer_id`, `status`, `placed_at`, **total** (sum of quantity × unit_price), **item_count**. Use this instead of re-aggregating `order_items`.

v_low_stock

Active products where `stock_qty` ≤ `reorder_at`, ordered by stock then name. Columns: `id`, `sku`, `name`, `category`, `stock_qty`, `reorder_at`.

v_daily_sales

Orders and revenue by UTC calendar day, excluding cancelled tickets. Columns: sale_date, orders, revenue.

5. Seed catalog

Products (SKU is the voice-agent lookup key)

SKU	Name	Category	Price	Reorder
ESP-HOT	Espresso	coffee	3.50	20
LAT-OAT	Oat Latte	coffee	5.75	15
CAP-CLS	Cappuccino	coffee	4.75	15
DRP-ETH	Ethiopia Pour Over	coffee	5.50	10
CLD-BRW	Cold Brew	coffee	5.00	12
TEA-EAR	Earl Grey	tea	3.75	8
TEA-JAS	Jasmine Green	tea	3.75	8
PST-CRO	Butter Croissant	pastry	4.25	8
PST-ALM	Almond Croissant	pastry	4.75	6
PST-MUF	Blueberry Muffin	pastry	3.50	6
FOD-BLT	BLT on Focaccia	food	9.50	5
FOD-AVO	Avocado Toast	food	8.75	5
MER-MUG	Harbor Mug	merchandise	18.00	4
MER-BGS	House Blend 12oz Beans	merchandise	16.00	6
MER-TEE	Harbor & Bean Tee	merchandise	28.00	3

On-hand stock in seed data varies; several pastries and beans start at or below reorder_at.

Customers (email is the order lookup key)

Name	Email	Phone
Maya Alvarez	maya@example.com	415-555-0101
Jonah Park	jonah@example.com	415-555-0102
Priya Shah	priya@example.com	415-555-0103
Eli Nguyen	eli@example.com	415-555-0104
Sofia Rossi	sofia@example.com	415-555-0105
Avery Cole	avery@example.com	415-555-0106
Kenji Watanabe	kenji@example.com	415-555-0107
Lila Brooks	lila@example.com	415-555-0108
Omar Haddad	omar@example.com	415-555-0109
Nina Volkov	nina@example.com	415-555-0110
Theo Marin	theo@example.com	415-555-0111

Name	Email	Phone
Harper Quinn	harper@example.com	415-555-0112

Employees

Name	Role	Rate
Sam Ortiz	manager	28.00
Riley Chen	shift_lead	22.50
Jordan Blake	barista	19.00
Amelia Diaz	barista	19.50
Noah Kim	baker	21.00
Casey Bell	barista	18.50

Shifts are relative to `CURRENT_DATE` (two days back through tomorrow). Stations: bar, register, kitchen, floor.

6. Voice agent tools

The Speech to Speech session exposes these functions. `query_database` is SELECT/WITH only, 4s timeout, max 50 rows. Writes always confirm with the operator first.

Tool	Purpose	Key arguments
<code>inspect_schema</code>	List tables/views and columns	optional <code>table_name</code>
<code>query_database</code>	Read-only SQL	<code>sql</code>
<code>create_customer</code>	Insert a loyalty guest	name, email, phone?
<code>create_order</code>	New ticket; decrements stock	customer_email, items[{product, quantity}], notes?, employee_name?
<code>update_order_status</code>	Move a ticket	order_id, status
<code>adjust_inventory</code>	Signed stock change + audit row	product, delta, reason

7. Example SQL

```
SELECT * FROM v_low_stock;
SELECT e.name, s.station, s.start_time, s.end_time FROM shifts s JOIN employees e
ON e.id = s.employee_id WHERE s.shift_date = CURRENT_DATE ORDER BY s.start_time;
SELECT c.name, SUM(t.total) AS spend FROM v_order_totals t JOIN customers c ON
c.id = t.customer_id WHERE t.status <> 'cancelled' AND t.placed_at >= NOW() -
INTERVAL '30 days' GROUP BY c.name ORDER BY spend DESC;
SELECT o.id, c.name, o.status, o.notes, t.total FROM orders o JOIN customers c ON
c.id = o.customer_id JOIN v_order_totals t ON t.order_id = o.id WHERE o.status NOT
IN ('completed', 'cancelled') ORDER BY o.placed_at;
```

Console shortcuts (after source `./env.sh`)

Command	Meaning
<code>python scripts/db_console.py</code>	Interactive voice=> prompt
<code>\tables / \views / \d products</code>	Catalog

Command	Meaning
\preview orders	First 20 rows
\low \open \shift \sales	Stock, open tickets, today's shifts, daily sales
python scripts/db_console.py -c "SQL"	One-shot query

Things to ask the voice agent

- What's low in stock?
- Who's on the bar today?
- What's revenue for the last seven days?
- Which tickets are still open?
- Who is the top customer by spend?
- Create an order for Maya: two oat lattes and a croissant.
- Mark the pending cold brew as ready.
- We wasted three almond croissants — take them off the shelf.

Regenerate this PDF: `python scripts/make_db_guide.py` (requires reportlab).